

CSCI 2406 | Computer Organization & Architecture | TOPIC 6

Data Transfer and Addressing

Load/store instructions, addressing modes,
memory alignment, arrays, and pointers

Dr. J. Burns

Topic 6 Outline

Topic Objective

Explain how AArch64 transfers values between registers and memory and how an instruction forms the effective address for a memory access.

- **Section 1:** Memory addresses and load/store semantics
- **Section 2:** Transfer width and extension
- **Section 3:** AArch64 addressing modes
- **Section 4:** Alignment and pair transfers
- **Section 5:** Summary and self-test

Section 1

Memory Addresses and Load/Store Semantics

AArch64 Is a Load/Store Architecture

Separation of Memory Access and Computation

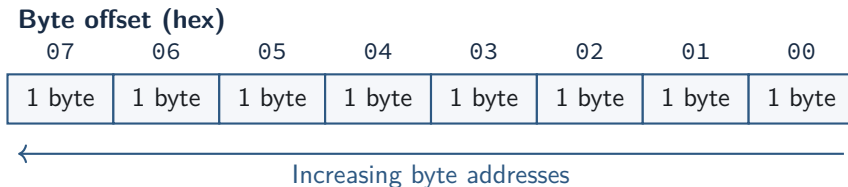
Arithmetic and logical instructions operate on register values. Dedicated load and store instructions transfer values between registers and memory.

```
ldr x1, [x0]
add x1, x1, #5
str x1, [x0]
```

- `ldr` reads a value from memory into a register.
- `add` changes the register value.
- `str` writes the register value back to memory.

Memory Is Byte Addressed

- Each memory address identifies one byte.
- Multi-byte values occupy consecutive addresses.
- The instruction determines how many bytes are transferred.



Example: One 64-Bit Value

Starting address: 1000 (hexadecimal). The rightmost byte is at 1000; the next byte to its left is at 1001; the leftmost byte is at 1007. Together, the eight bytes occupy 64 bits.

Bracket Notation Means Memory Access

Register Operand

`add x0, x1, x2`

- Uses values already in registers.
- No data-memory operand appears.

Memory Operand

`ldr x0, [x1]`

- x1 contains an address.
- Brackets mean access memory at that address.

Do Not Confuse Address and Data

The value in x1 is the address. The bytes stored at that address are the data transferred by the load or store.

Loading Data with `ldr`

64-Bit Load

```
ldr x0, [x1]
```

- The effective address is the 64-bit value in `x1`.
- Eight bytes are read from memory.
- The 64-bit value is written into `x0`.

32-Bit Load

`ldr w0, [x1]` reads four bytes into `w0`. Any write to `w0` clears bits `[63:32]` of the corresponding `x0` register.

Storing Data with `str`

64-Bit Store

```
str x0, [x1]
```

- The effective address is the value in `x1`.
- The eight-byte value in `x0` is written to memory.
- The source register is not changed by the store.

Narrow Stores

`strb` writes the low 8 bits of a register, and `strh` writes the low 16 bits. `str w0, [x1]` writes the low 32 bits.

Section 2

Transfer Width and Extension

Load Width Determines Bytes Read

Instruction	Bytes Read	Destination Result
<code>ldrb w0, [x1]</code>	1 byte	Zero-extended to 32 bits in w0
<code>ldrh w0, [x1]</code>	2 bytes	Zero-extended to 32 bits in w0
<code>ldr w0, [x1]</code>	4 bytes	Written to w0; upper half of x0 becomes zero
<code>ldr x0, [x1]</code>	8 bytes	Full 64-bit value written to x0

Width Comes from the Instruction

A memory address does not carry a type. The load/store instruction determines the number of bytes transferred.

Signed Loads Preserve Negative Values

Sign Extension

A signed load copies the sign bit of the narrow source into the new upper bits so the two's-complement value is preserved.

Instruction	Source Width	Result
<code>ldrsb x0, [x1]</code>	8 bits	Sign-extended to 64 bits
<code>ldrsh x0, [x1]</code>	16 bits	Sign-extended to 64 bits
<code>ldrsw x0, [x1]</code>	32 bits	Sign-extended to 64 bits

Example

If memory contains the byte `0xff`, `ldrb` interprets the transferred bits as 255 after zero extension, while `ldrsb x0, [x1]` produces the 64-bit representation of `-1`.

Load Width and Extension Example

Assume `x1 = 0x2000` and memory at that address contains the byte `0x80`.

- **Unsigned byte load:** `ldrb w0, [x1]`
 - Reads one byte and fills the remaining bits with zeros.
 - Result: `w0 = 0x00000080`, decimal value 128.
 - Writing `w0` also clears the upper 32 bits of `x0`.
- **Signed byte load:** `ldrsb x0, [x1]`
 - Reads one byte and copies its sign bit into all higher bits.
 - Result: `x0 = 0xffffffffffff80`, decimal value -128 .

Section 3

AArch64 Addressing Modes

Three Addressing Types

A **pointer** holds a memory address. In these examples, the **base register** holds that pointer: C pointer `ptr` corresponds to register `x1`.

❶ **Offset:** access an address relative to the base; leave the base unchanged.

- `ldr x0, [x1, #8]` C: `value = ptr[1];`

❷ **Pre-index:** adjust the pointer, then access that address.

- `ldr x0, [x1, #8]!` C: `ptr++; value = *ptr;`

❸ **Post-index:** access the current address, then adjust the pointer.

- `ldr x0, [x1], #8` C: `value = *ptr; ptr++;`

C Conventions Used in This Section

`uint64_t *ptr` corresponds to `x1`; `uint64_t value` corresponds to `x0`. Each element occupies 8 bytes, so `ptr++` advances by 8 bytes. Examples assume valid array accesses.

The Effective Address

Definition

The effective address is the memory address computed for a particular load or store instruction.

- A base register holds the pointer that supplies the starting address.
- An optional immediate or register offset can modify that address.
- Some addressing forms also write an updated address back to the base register.

Base Register

AArch64 memory operands normally use a 64-bit general-purpose register or `sp` as the base. A `w` register is not used as the base address register.

1. Offset: Base Register Only

Syntax

```
ldr x0, [x1]
```

- Effective address = x1.
- x1 is unchanged.
- This corresponds closely to dereferencing a pointer already holding the required address.

C Analogy

```
value = *ptr;
```

This is offset addressing with a zero offset: read the current element and leave ptr unchanged.

1. Offset: Immediate Offset

Example

```
ldr x0, [x1, #16]
```

- Effective address = $x1 + 16$.
- The base register $x1$ is unchanged.
- Useful for structure fields and fixed-position array elements.

Equivalent C

```
value = ptr[2];
```

Read the element 16 bytes beyond `ptr`; `ptr` is unchanged.

Encoding Detail

A64 has more than one load/store immediate encoding. Common unsigned offsets are scaled by transfer size; unscaled forms use a smaller signed byte offset.

1. Offset: Register Offset

Pointer Access: x2 Holds a Byte Displacement

```
ldr x0, [x1, x2]
```

```
value = *(ptr + offset);
```

Here ptr is a pointer to 64-bit elements. C counts offset in elements; x1 holds ptr and x2 holds offset * 8 bytes.

Array Index: x2 Holds Element Index i

```
ldr x0, [x1, x2, lsl #3]
```

```
value = ptr[i];
```

Multiply i by 8 to obtain the byte displacement for 64-bit elements.

Both forms leave x1 (the pointer ptr) unchanged.

2. Pre-Indexed Addressing

Assembly

```
ldr x0, [x1, #8]!
```

Equivalent C: Advance, Then Read

```
ptr++;
```

```
value = *ptr;
```

Or, in one expression: `value = *++ptr;`

- Access address: original $x1 + 8$ (the next element).
- Base after the instruction: original $x1 + 8$.
- The exclamation mark indicates writeback; A64 pre-index uses an immediate offset.

3. Post-Indexed Addressing

Assembly

```
ldr x0, [x1], #8
```

Equivalent C: Read, Then Advance

```
value = *ptr;
```

```
ptr++;
```

Or, in one expression: `value = *ptr++;`

- Access address: original x1 (the current element).
- Base after the instruction: original x1 + 8.
- The immediate outside the brackets specifies the pointer update.

Addressing Mode Comparison

Form	Example	Access Address	Base After
Offset (zero)	<code>ldr x0, [x1]</code>	$x1$	unchanged
Immediate offset	<code>ldr x0, [x1, #16]</code>	$x1 + 16$	unchanged
Register offset	<code>ldr x0, [x1, x2]</code>	$x1 + x2$	unchanged
Pre-index	<code>ldr x0, [x1, #8]!</code>	$x1 + 8$	$x1 + 8$
Post-index	<code>ldr x0, [x1], #8</code>	$x1$	$x1 + 8$

Key Distinction

Offset addressing computes an access address without changing the base register; pre-indexed and post-indexed forms also write an updated address back to the base.

Byte Access: Advance by One Byte

C: Read One Byte, Then Advance

```
uint8_t *ptr;  
value = *ptr++;
```

AArch64: x0 Holds ptr

```
ldrb w1, [x0], #1
```

- Read one byte and zero-extend it into w1.
- Advance by 1 byte for a byte array, compared with 8 bytes for a 64-bit array.
- Match both the load width and pointer increment to the element size.

Section 4

Alignment and Pair Transfers

Natural Alignment

Natural Alignment Rule

An n -byte object is naturally aligned when its starting address is a multiple of n .

- 2-byte halfword: address divisible by 2.
- 4-byte word: address divisible by 4.
- 8-byte doubleword: address divisible by 8.

AArch64 Qualification

Many ordinary AArch64 loads and stores to normal memory permit unaligned addresses, but natural alignment is preferred. Misaligned accesses may be slower or cross memory-system boundaries, and some access classes have stricter requirements.

Natural Alignment Boundaries

Byte offset (hex)	07	06	05	04	03	02	01	00
8-bit (1 byte)								
16-bit (2 bytes)	2 bytes		2 bytes		2 bytes		2 bytes	
32-bit (4 bytes)	4 bytes				4 bytes			
64-bit (8 bytes)	8 bytes							

Aligned Starting Addresses

Assume offset 00 begins at address 1000 (hex). Read each group from its right edge: bytes can start anywhere; 16-bit objects at offsets 00, 02, 04, 06; 32-bit objects at 00, 04; and a 64-bit object at 00. The pattern repeats every 8 bytes.

Load and Store Pair: `ldp` and `stp`

Store Pair

```
stp x0, x1, [x2]
```

- Stores two register values.
- The memory locations are adjacent.

Load Pair

```
ldp x0, x1, [x2]
```

- Loads two register values.
- The memory locations are adjacent.

Important Qualification

A pair transfer is one architectural instruction, but it is not guaranteed to be atomic or to execute as one physical memory-bus transaction.

Section 5

Summary and Self-Test

Topic 6 Summary

- AArch64 uses explicit load/store instructions to move values between registers and memory.
- The instruction determines transfer width and whether a narrow loaded value is zero-extended or sign-extended.
- The base register holds a pointer. Offset addressing leaves it unchanged; pre-index and post-index addressing update it before or after the access, respectively.
- Scaled register offsets map naturally to array indexing; post-indexing maps naturally to pointer traversal.
- Natural alignment is preferred, although many ordinary unaligned accesses to normal memory are permitted.
- `ldp` and `stp` transfer adjacent register pairs without implying atomicity or a particular physical bus transaction.

Self-Test 1/2

- 1 If C pointer `ptr` is held in base register `x1`, what C statement corresponds to `ldr x0, [x1]`? What do the brackets mean?
- 2 How many bytes are read by `ldrb`, `ldrh`, `ldr w0, [...]`, and `ldr x0, [...]`?
- 3 Memory contains byte `0x80`. What decimal values result from `ldrb w0, [x1]` and `ldrsb x0, [x1]`? Explain the extension used.
- 4 Name the three addressing types. Which leave the base pointer unchanged, and which update it?
- 5 For `ldr x0, [x1, #16]`, give the effective address and equivalent C statement, assuming `ptr` points to 64-bit elements.

Self-Test 2/2

- 6 Translate `value = ptr[i];` into a load using `x1` for `ptr` and `x2` for `i`. Assume 64-bit elements. Why is a shift of 3 needed?
- 7 Compare `ldr x0, [x1, #8]!` and `ldr x0, [x1], #8`. Give the access address, final base value, and equivalent C statements for each.
- 8 For a byte pointer, translate `value = *ptr++;` using `x0` for `ptr`. Why is the increment 1 rather than 8?
- 9 What does natural alignment mean for an 8-byte object? Must every ordinary AArch64 load use a naturally aligned address?
- 10 What do `ldp` and `stp` transfer? Do they guarantee an atomic transfer?

Branching and Iteration

Topic 7 develops the comparison, branching, and control-flow instructions used to implement decisions and repetition.

- Comparison instructions
- Conditional and unconditional branches
- Loops and selection structures
- Translation of high-level control flow